

Büyük Dil Modeli (LLM) Uygulamalarında Değerlendirme (Evaluation)

LangChain ile Soru-Cevap (QA) ve Hata Ayıklama (Debugging) Süreçlerinin Otomasyonu

1. Giriş ve Değerlendirme Problemi

Büyük Dil Modelleri (LLM) ile karmaşık uygulamalar geliştirirken karşılaşılan en kritik ve zorlu adımlardan biri, uygulamanın ne kadar iyi performans gösterdiğini değerlendirmektir. Modelin doğruluk kriterlerini karşılayıp karşılamadığını bilmek, geliştirme sürecinin temelini oluşturur. Sistemde herhangi bir değişiklik yapıldığında (örneğin kullanılan dil modelini değiştirmek, RAG / Vektör veri tabanından veri getirme stratejilerini güncellemek veya parametreleri ayarlamak), bu değişikliğin sistemi daha iyiye mi yoksa kötüye mi götürdüğünü objektif olarak ölçmek gerekir.

Günlük Hayattan Analoji (Şefin Çorbası): Bir restoranda yeni bir çorba tarifi geliştiren bir şef düşünün. Şef, çorbaya her yeni baharat eklediğinde (model/parametre değişikliği) tadına bakarak yemeğin gelişip gelişmediğini test eder. Ancak restoranda her gün yüzlerce kase çorba servis ediliyorsa ve şef her kaseyi tek tek el ile tatmak zorunda kalırsa (manuel değerlendirme), bu süreç bir süre sonra tıkanır. Ayrıca şefin tat alma duyusu gün içinde değişebilir (subjektif değerlendirme). Bu nedenle şefe, her kasedeki malzeme dengesini ve tadı anlık, tutarlı ve otomatik olarak ölçecek laboratuvar testleri (otomatik değerlendirme) gereklidir.

2. Değerlendirme Veri Seti Hazırlama

Bir LLM uygulamasını test etmek için öncelikle değerlendirme yapabileceğimiz veri noktalarına (soru ve doğru cevap çiftleri) ihtiyacımız vardır. Raporumuza konu olan eğitimde, bu veri noktalarını oluşturmak için iki farklı yöntem ele alınmıştır:

A. Manuel (El İle) Veri Kümesi Oluşturma

Bu en temel yöntemde, verileri bizzat inceleyerek örnek sorular ve bunların karşılığında kesin doğru kabul edilen 'gerçek cevap' (ground truth) çiftlerini el ile yazarız. Örneğin, ürün kataloğundaki belgeleri inceleyerek şu şekilde test çiftleri oluşturabiliriz:

```
# Manuel olarak değerlendirme örnekleri oluşturma examples = [ { "query": "Cozy Comfort Pullover Set ürününün yan cepleri var mı?", "answer": "Evet" }, { "query": "Bu ceket hangi koleksiyondan?", "answer": "DownTek koleksiyonu" } ]
```

B. LLM ile Otomatik Soru-Cevap Üretimi

Manuel yöntem küçük ölçekli testler için ideal olsa da yüzlerce doküman içeren büyük sistemlerde ölçeklenemez ve aşırı zaman alır. Bu süreci otomatikleştirmek için dil modellerinin kendisinden yararlanabiliriz. LangChain içerisinde yer alan **QA Generation Chain** (Soru-Cevap Üretim Zinciri), dokümanları girdi olarak alır ve her dokümandan otomatik olarak soru ve doğru cevap ikilileri üretir.

Kodun çıktıları sözlük (dictionary) yapısında almak için `apply_and_parse` yöntemi kullanılır, böylece her öge doğrudan değerlendirme sistemine beslenebilir.

```
from langchain.evaluation.qa import QAGenerateChain from langchain.chat_models import ChatOpenAI #  
Otomatik soru-cevap çifti üreten zincirin kurulması example_gen_chain =  
QAGenerateChain.from_llm(ChatOpenAI()) # Dokümanlardan otomatik test verisi türetme (Sözlük yapısında  
parse edilir) new_examples = example_gen_chain.apply_and_parse( [{"doc": doc} for doc in data[:5]] )  
# Manuel ve otomatik örnekleri birleştirme examples.extend(new_examples)
```

3. langchain.debug ile Adım Adım Hata Ayıklama

LLM uygulamaları arka arkaya bağlanan birçok farklı zincirden ve adımdan oluşur. Sadece nihai cevaba bakmak, sistemin nerede hata yaptığını anlamak için yetersizdir. Hatalı bir çıktının sebebi dil modelinin kendisi olabileceği gibi, çoğunlukla **veri getirme (retrieval)** adımında yanlış dokümanların çekilmiş olmasından kaynaklanır. LangChain geliştiricilere bu adımları görünür kılmak için çok güçlü bir araç sunar: **langchain.debug = True**

```
import langchain # Hata ayıklama modunu aktif etme langchain.debug = True # debug aktifken bir sorgu
çalıştırma qa.run(examples[0]["query"])
```

Hata Ayıklama Modunda Zincirlerin İzlenmesi: Bu mod aktif edildiğinde, sistem arka planda çalışan tüm katmanları sırasıyla ekrana döker: 1. İlk olarak ana zincir olan **RetrievalQA** başlatılır. 2. Doküman birleştirme yöntemi olan **StuffDocumentsChain** adımına geçilir. 3. Alt tarafta **LLMChain** çalıştırılır ve arama adımından getirilen ham dokümanlar (context) ve kullanıcının asıl sorusu sisteme parametre olarak verilir. 4. Son adımda, **ChatOpenAI** modeline giden nihai sistem talimatı (System Message) görüntülenir. Bu talimatta modelin uydurma cevap vermesini engelleyen 'Eğer cevabı bilmiyorsan, bilmediğini söyle, cevap uydurmaya çalışma' direktifinin sisteme nasıl eklendiği açıkça görülür. 5. Çıktıda ayrıca token tüketimi (prompt_tokens, completion_tokens, total_tokens) gibi kritik metrikler yer alır. Bu metrikler, uygulamanın maliyetini optimize etmek için doğrudan takip edilmelidir.

4. LLM Destekli Otomatik Değerlendirme (LLM-as-a-Judge)

Oluşturduğumuz test senaryolarını tek tek gözle kontrol etmek sürdürülebilir değildir. Ancak klasik yazılım testlerindeki gibi birebir metin eşleştirmesi (string matching veya regex) yapmak da imkansızdır. Çünkü dil modelleri açık uçlu metinler üretir ve kelimeler değişse de anlam aynı kalabilir. Buradaki en modern ve popüler çözüm, **bir dil modelini diğer dil modellerini ve zincirlerini değerlendirmek için hakem (evaluator) olarak kullanmaktır.**

```
from langchain.evaluation.qa import QAEvalChain from langchain.chat_models import ChatOpenAI # Tüm
test verileri için tahminler oluşturma predictions = qa.apply(examples) # Değerlendirme (eval)
zincirinin oluşturulması ve çalıştırılması eval_chain = QAEvalChain.from_llm(ChatOpenAI())
graded_outputs = eval_chain.evaluate(examples, predictions)
```

Semantik (Anlamsal) Değerlendirmenin Önemi

Sistemin neden klasik eşleştirmelerle test edilemeyeceğini şu somut örneklerle görebiliriz:

Soru / Sorgu	Gerçek Cevap (Ground Truth)	Modelin Tahmini	Sistem Kararı (Grade)
Cozy Comfort Pullover Set ürününün yan cepleri var mı?	Evet	Cozy Comfort Pullover Set, Stripe modelinde yan cepler bulunmaktadır.	DOĞRU (CORRECT)

Analiz: Yukarıdaki örnekte görüldüğü üzere, gerçek cevap sadece tek bir kelime ('Evet') iken, modelin tahmini oldukça uzun ve detaylı bir cümledir. 'Evet' kelimesi modelin tahmin metni içinde birebir geçmemektedir. Klasik yöntemler bu testi doğrudan 'BAŞARISIZ' sayacakken, LangChain tarafından kullanılan **QAEvalChain** (arka planda çalışan LLM), iki metnin anlamsal olarak tamamen aynı doğrultuda olduğunu kavrayarak bu sonuca **'DOĞRU' (CORRECT)** notunu verir.

Geleneksel Yazılım Testi ve LLM Yardımlı Değerlendirme Farkları

Kriter	Geleneksel Yöntem (Regex / Metin Eşleşmesi)	LLM Yardımlı Yöntem (Anlamsal Değerlendirme)
Doğrulama Biçimi	Karakter bazlı tam eşleşme, anahtar kelime veya düzenli ifadeler aranır.	Metinlerin arkasındaki anlamsal (semantik) niyet ve bağlam çözülür.
Esneklik Derecesi	Çok katıdır. En ufak bir eş anlamlı kelime kullanımında testi başarısız sayar.	Yüksek esneklik. Tamamen farklı kelimelerle kurulmuş aynı anlamdaki cümleleri tanır.
Ölçeklenebilirlik	Her bir olası doğru yanıt varyasyonunun el ile tanımlanması gerekir.	Sınırsız. Serbest biçimli ve ucu açık her türlü LLM çıktısını analiz edebilir.

5. LangChain Değerlendirme Platformu ve Değerlendirme Çarkı

Geliştirilen bu testlerin yerel bilgisayarda kalması yerine, kalıcı hale getirilmesi ve bir arayüzde izlenmesi büyük önem taşır. LangChain Değerlendirme Platformu (örneğin eğitimdeki 'deeplearningai' oturumu), kod seviyesinde yapılan tüm test çalıştırmalarını görselleştirir. Arayüzün sunduğu avantajlar şunlardır: • **Görsel Hata Ayıklama:** Kodda debug = True yapıldığında ekrana gelen tüm zincir ve token bilgilerini çok daha okunabilir, hiyerarşik bir arayüz ile sunar. • **Değerlendirme Çarkı (Evaluation Flywheel):** Canlı ortamdan veya testlerden gelen soru-cevap çiftleri, arayüzdeki tek bir buton yardımıyla değerlendirme veri setlerine (örn. 'deep learning' veri kümesi) eklenebilir. Bu sayede sistem çalıştıkça test veri kümesi sürekli büyür, zenginleşir ve her güncellemede uygulamanın kalitesi daha güvenli şekilde test edilir.